

The State of Sparsity in Large Language Models

Anonymous Authors¹

Abstract

Pruning large language models (LLMs) has exhibited markedly different behavior compared to pruning more traditional computer vision models, shaping several predominant contemporary views regarding its supposed nature. However, recent evidence has begun to challenge these commonly held assumptions on multiple fronts, raising questions about whether the current state of sparsity in LLMs indeed reflects intrinsic limitations or is instead a result of methodological choices and experimental conventions. Motivated by this discrepancy, we conduct a systematic study of progressive sparse training frameworks for pre-trained LLMs, with particular attention to the optimization components accompanying sparsification. Our analysis reveals that several factors, often examined independently in prior work, play a central role in enabling stable and reliable sparse training. When these components are jointly considered, we find that sparsification can be carried out in a controlled manner at regimes well beyond those commonly explored, without severe degradation in perplexity or downstream task accuracy. This leads to consistent improvements over existing approaches, setting a new state of the art in extreme LLM sparsity.

1. Introduction

As large language models (LLMs) continue to scale, their deployment incurs substantial compute and memory costs (Brown et al., 2020). Pruning is a critical strategy to mitigate these overheads, offering the potential to drastically reduce model size and accelerate inference (Hoefer et al., 2021; Agarwalla et al., 2024).

A prevailing idea present in transformer-based LLM pruning is that it exhibits behavior that differs markedly from

that observed in more traditional computer vision models, such as ineffectiveness magnitude based pruning (Frantar & Alistarh, 2023), failure of global comparison grouping due to the imbalance distribution (Sung et al., 2023) unlike convolution networks, and importance of parameters being highly sensitive to activation outliers (Sun et al., 2024). This perceived discrepancy has shaped prevailing views that LLMs are inherently less amenable to aggressive sparsification, with reliable performance often believed to be attainable only within relatively moderate sparsity regimes around 50~70%. As a result, much of the existing literature has implicitly treated these limitations as intrinsic properties of LLMs, rather than outcomes contingent on how sparsification is performed (Frantar & Alistarh, 2023; Sun et al., 2024; Agarwalla et al., 2024).

However, recent work has provided indications that these commonly held assumptions may warrant reevaluation on multiple fronts (Shin et al., 2024; Agarwalla et al., 2024; Lee et al., 2026). In particular, an emerging body of evidence suggests that modern LLMs may remain functional at sparsity levels beyond the moderate regimes often treated as a practical ceiling, when pruning is accompanied by appropriate procedures (Kurtic & Alistarh, 2022; Agarwalla et al., 2024); notably, Lee et al. (2026) reports functional performance at 90% sparsity.

This raises concerns that the limitations implied by prior works may be contingent on methodological choices rather than intrinsic constraints. Indeed, much of the literature proceeds from the premise that LLMs are too large for extensive retraining or broad hyperparameter exploration, and the resulting one-shot protocols may not provide a sufficiently diagnostic setup for constructing a balanced perspective on the state of sparsity on LLMs.

In this work, we aim to provide a methodological foundation for reassessing these perceptions. Building on gradual pruning under the progressive sparse training framework, we decompose the procedure into core strategic components and systematically examine alternative designs, thereby exploring regions of the design space that prior idea-driven studies may have overlooked. This reveals that the absence of optimization strategies to account for progressive network sparsity constitutes a major failure mode, leading to loss spikes or severe under-training. Further investigation sug-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

gests that these can be mitigated via improved loss-aware pruning saliency and learning-rate scheduling coordinated with sparsity progression.

These insights leaves us with set of effective strategies that integrate second-order saliency with gradual sparsity schedules and stable optimization techniques, where we demonstrate that they are essential for preserving perplexity and downstream accuracy under aggressive sparsification. Surprisingly, these strategies alone demonstrates a capacity to set a new state of the art among existing strategies for pruning standard language model: LLaMA-2-7B to extreme sparsity levels, achieving perplexities of 13.48 and 19.47 at 95% and 99% sparsity on WikiText-2. We further show that such extreme sparsity obtained as a result of these strategies enables pruning to deliver substantial practical acceleration for LLMs, unlocking $\times 3.22$ decoding speedups and $\times 6.21$ memory savings compared to dense counterpart.

Our contributions are summarized as follows:

- Systematic analysis of sparsification strategies. We ablate the techniques required to reach extreme sparsity and distill a practical recipe for stable high-sparsity pruning.
- State-of-the-art high-fidelity sparsity. We prune 7B-scale models up to 99% sparsity while maintaining comparative performance, updating the current state of sparsity in LLMs while opening a new avenue to making extreme sparsity substantially more practical.
- Practical efficiency. We demonstrate to translate extreme sparsity into tangible benefits, achieving $3.22\times$ decoding speedup and $6.21\times$ memory savings using state-of-the-art sparse kernels.

2. Background

2.1. Pruning

Pruning aims to find sparse neural networks whose weight matrices are mostly populated by zeros, so as to skip corresponding computation and memory allocation while preserving task performance (Hoefler et al., 2021). In principle, pruning can facilitate more aggressive compression than structured alternatives such as quantization, knowledge distillation, or structured sparsity (Agarwalla et al., 2024), as it selectively retain individual weights rather than being constrained by predefined structural patterns.

A long line of studies has demonstrated the utility of pruning for compressing neural networks, with early success dating back to Janowsky (1989); Mozer & Smolensky (1988); LeCun et al. (1989); Hassibi & Stork (1992); Hagiwara (1994); Beck & Teboulle (2009). Pruning has garnered renewed attention in the past decade following Han et al. (2015), as widespread adoption of deep neural network for complex tasks having simultaneously introduced prohibitive computational costs.

Further efforts have identified various effective strategies and heuristics that improve achievable compression rates. Early studies showed that iterative pruning, when combined with retraining, can gradually remove redundant parameters while preserving accuracy (Zhu & Gupta, 2017; Singh & Alistarh, 2020). Other works demonstrated that allowing the sparsity pattern to dynamically evolve during training enables models to reallocate parameters toward more informative connections and often match the performance of dense baselines at comparable computational budgets (Mocanu et al., 2018; Peste et al., 2021). Beyond pruning schedules, advances in saliency estimation further improved robustness to pruning by leveraging gradient- or sensitivity-based criteria, enabling more accurate identification of dispensable parameters (Lee et al., 2019; Sanh et al., 2020).

2.2. Pruning in LLMs

Despite this, pruning has yet to gain broad adoption in practical LLM workflows, in part due to technical and implementation challenges that complicate its integration relative to more structured alternatives (Kim et al., 2024; Macko & Boža, 2025). However, continued progress in recent years has made its benefits increasingly practical, positioning pruning as a promising avenue for achieving compression levels beyond the structural limits of these alternatives (Agarwalla et al., 2024).

Early efforts in LLM pruning largely focused on developing computationally inexpensive pruning procedures, reflecting early reactions to the rapid emergence of extreme scaling trends that were perceived to render large language models inaccessible under conventional system constraints. Post-training layer-wise one-shot methods (Frantar & Alistarh, 2023; Sun et al., 2024) exemplified this direction by demonstrating that moderate sparsity could be obtained with minimal pruning cost, data requirements, or system overhead. While these methods established the feasibility of pruning in smaller systems, their reliance on simplifying assumptions limited their effectiveness in aggressive compression settings that are more directly tied to inference-time deployment costs in large-scale production systems (Shin et al., 2024; Lee et al., 2026).

More recent work has tried to shift the focus toward extending the utility of pruning to these cost-critical large systems, exploring regimes of substantially higher sparsity and more adaptive pruning strategies (Agarwalla et al., 2024; Lee et al., 2026). These efforts had demonstrated that one can achieve beyond 70~90% sparsity without catastrophic performance degradation (Agarwalla et al., 2024; Lee et al., 2026). This has been supported by concurrent advancement in kernel software and hardware, demonstrating that near-ideal acceleration can be realized (Agarwalla et al., 2024; Macko & Boža, 2025).

2.3. Realizing Sparsity Benefits

Realizing both theoretical memory savings and wall-clock acceleration on GPUs remains non-trivial due to irregular memory access patterns that conflict with dense hardware parallelism, unless additional hardware-aligned structured constraints (e.g., N:M semi-structured (Zhou et al., 2021), block-structured (Gray et al., 2017)) are imposed on the sparsity pattern, which in turn restricts the space of effective patterns that can be explored (Mishra et al., 2021).

However, recent advancements are steadily bridging this hardware-software gap. Specialized kernel libraries focus on improving Sparse Matrix-matrix Multiplication (SpMM) (Gale et al., 2020; Xia et al., 2023; Fan et al., 2025) and Sparse Matrix-Vector multiplication (SpMV) (Lu & Liu, 2023; Macko & Boža, 2025) by utilizing optimized load-balancing, scheduling mechanisms, and novel storage formats to enable practical speedups on GPUs. Furthermore, dedicated architectures like the Cerebras Wafer-Scale Engine (Lie, 2023; Agarwalla et al., 2024) exploit immense memory bandwidth and dataflow scheduling to process linear algebra at the vector-scalar granularity, allowing the hardware to natively skip zero-valued operands. These developments suggest that as the inference stack matures, the ability to induce high sparsity without performance degradation—as explored in this work—becomes increasingly critical for unlocking real-world efficiency.

3. Analysis

Let us start by formulating pruning as constrained risk minimization problem:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) \quad \text{s.t.} \quad \|\mathbf{w}\|_0 \leq k, \quad (1)$$

where we minimize the risk $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$ through optimizing the model parameters $\mathbf{w} \in \mathbb{R}^d$ with $k < d$ number of non-zero weights computed by the L_0 -norm $\|\cdot\|_0$.

While there exists various strategies (Hoeffler et al., 2021) to solve Equation (1) (Zhu & Gupta, 2017; Frankle & Carbin, 2018; Evci et al., 2020; Liu et al., 2020), we specifically aim to analyze atop progressive sparse training framework¹ (Zhu & Gupta, 2017), simple yet effective pruning strategy that has historically proven effective in revealing key components governing sparsification performance, thus a common basis for various analyses (Frankle & Carbin, 2018) and for developing advanced sparsification procedure (Evci et al., 2020; Kurtic & Alistarh, 2022). Formally, we decompose

¹Commonly referred to as gradual pruning or iterative pruning.

the pruning process as:

$$\mathbf{m}^{n+1} = \mathcal{M}^n(\mathbf{m}^n \odot \mathbf{w}^n) \quad (2)$$

$$\text{s.t.} \quad d = \|\mathbf{m}^0\|_0 \geq \dots \geq \|\mathbf{m}^N\|_0 = k \quad (3)$$

$$\mathbf{w}^{n+1} = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{m}^{n+1} \odot \mathbf{w}) \quad (4)$$

where we alternate between the masking operation $\mathcal{M}^n : \mathbb{R}^d \rightarrow \{0, 1\}^d$ in Equation (2) with sparsity schedule in Equation (3) for the pruning mask $\mathbf{m} \in \{0, 1\}^d$ and the weight training for remaining parameters in Equation (4). The process is initialized with the pretrained weights, setting $\mathbf{w}^0 = \mathbf{w}_{\text{pre}}$ with a dense starting mask $\mathbf{m}^0 = 1$.

In this section, we investigate these procedures through designing these three core components:

- **Masking strategy** (2) by designing saliency criteria and to determine which specific parameters in $\mathbf{w}$ to remove, and the sparsity allocation strategy with given saliency criteria (Section 3.1),
- **Sparsity schedule** (3) that would govern the progression of $\|\mathbf{w}\|_0$ towards the target k (Section 3.2), and
- **Training strategy** (4) to optimize the remaining active parameters in $\mathbf{w}$ for recovery (Section 3.3).

By analyzing the distinct impact of each component, we propose an optimized recipe for achieving highly sparse LLMs.

3.1. Masking Strategy

The ideal design for the masking function (2) would be one that would be closest to solving the subproblem $\arg \min_{\|\mathbf{m}\|_0 \leq k} \mathcal{L}(\mathbf{m} \odot \mathbf{w}^n)$. A common heuristic is to design a saliency criteria (Hoeffler et al., 2021), relying on a scalar importance score s_i to each parameter w_i based on the expected increase in loss $\mathcal{L}$ upon its removal (Han et al., 2015; Lee et al., 2019; LeCun et al., 1989).

In this subsection, we categorize standard criteria into three levels of approximation—magnitude, first-order, and second-order—and subsequently propose an efficient, optimizer-aware proxy designed for scalable LLM pruning.

3.1.1. SALIENCY DESIGN

Magnitude-based Saliency The most straightforward approach assumes that parameters with small absolute values are redundant (Han et al., 2015). This yields a magnitude score:

$$s_i^{\text{mag}} = |w_i|. \quad (5)$$

This heuristic relies on the assumption that that two parameters with the same magnitude are equally important to the objective function. Due to its computational efficiency and simplicity, magnitude pruning has become a ubiquitous baseline in the literature (Han et al., 2015; Zhu & Gupta,

2017; Evci et al., 2020). However, because weight magnitude is a poor proxy for the sensitivity of the loss $\mathcal{L}$, it becomes an unreliable predictor of parameter importance at higher sparsity and often leads to catastrophic degradation under one-shot pruning of LLMs (Frantar & Alistarh, 2023; Sun et al., 2024; Meng et al., 2024).

First-order Saliency To better incorporate sensitivity, first-order methods estimates the loss change induced by parameter removal using local first-order approximations of the objective (Lee et al., 2019). Under a small perturbation $\Delta w_i = -w_i$, the resulting loss change can be approximated as

$$\Delta \mathcal{L}_i \approx -g_i w_i, \quad (6)$$

where $g_i = \frac{\partial \mathcal{L}}{\partial w_i}$. This leads to the first-order saliency score

$$s_i^{1st} = |g_i w_i|. \quad (7)$$

This criterion, often termed ‘‘connectivity sensitivity’’, measures the immediate impact of a parameter on the loss. However, since the gradients of well-optimized pretrained models are often close to zero (LeCun et al., 1989), these salien- cies can limit their effectiveness in post-training pruning settings.

Second-order Saliency To overcome the limitations of first-order methods, second-order criteria (LeCun et al., 1989) incorporate curvature via the Taylor expansion: $\Delta \mathcal{L} \approx g^\top \Delta w + \frac{1}{2} \Delta w^\top H \Delta w$. Assuming convergence ($g \approx 0$) and a diagonal Hessian, the cost of removing w_i simplifies to:

$$s_i^{2nd} = \frac{1}{2} H_{ii} w_i^2 \quad (8)$$

While theoretically precise, second-order saliency requires computing the Hessian of the large-scale network, which remains computationally intractable.

Optimizer-aware Proxy To circumvent the cost of exact Hessian computation, we approximate the Hessian via the Generalized Gauss-Newton matrix (Botev et al., 2017), which coincides with the Fisher Information Matrix under standard cross-entropy objective (Martens, 2020). For converged models, this is estimated using the Empirical Fisher $F = \mathbb{E}[gg^\top]$ (Kunstner et al., 2019). In practice, its diagonal elements F_{ii} serve as a computationally efficient proxy for curvature and are computed as the expected squared gradients: $F_{ii} = \mathbb{E}[g_i^2]$.

However, explicitly computing F_{ii} still incurs non-negligible overhead, as it necessitates collecting and averaging gradients over data samples, a process that becomes a bottleneck in the context of LLM pruning.

To address this efficiency bottleneck, we leverage the running statistics inherently maintained by the Adam optimizer

(Kingma, 2014) during the retraining phase. Adam tracks the exponential moving average of squared gradients, denoted as $\hat{v}^t$, which effectively serves as an on-the-fly estimate of the diagonal Fisher information. By repurposing $\hat{v}^t$ as a proxy for the curvature term, we derive the following zero-cost saliency scores:

$$s_i^{\text{opt}-2} = \frac{1}{2} \hat{v}_i^t w_i^2 \quad (9)$$

This formulation enables dynamic, curvature-aware pruning without any additional computational overhead for gradient collection. Ablation studies in Appendix C.1 confirm that these optimizer-derived estimates are indistinguishable from exact on-the-fly derivatives.

3.1.2. SALIENCY COMPARISON

Setup To enable granular analysis of saliency criteria prior to scaling, we conduct our ablation studies on the smaller-scale OPT-125M (Zhang et al., 2022) using the SlimPajama dataset (Soboleva et al., 2023). To ensure a fair comparison with the optimizer-aware approach, we also utilize the first momentum state $\hat{m}_i^t$ maintained by Adam for the first-order saliency baseline. We employ a gradual sparsity schedule coupled with learning rate warm-up; the specific impacts of these scheduling components and the learning rate warm-up are analyzed independently in subsequent sections. Detailed training and evaluation setup can be found at Appendix B.2.

Table 1. Effect of saliency and pruning comparison group across sparsity levels on OPT-125M.

Saliency	Group	Sparsity					
		50%		70%		90%	
		Wiki	C4	Wiki	C4	Wiki	C4
Magnitude	Layer	27.89	27.68	32.44	31.29	47.62	42.28
	Global	28.83	34.54	34.54	31.41	54.79	46.43
1st-order	Layer	30.81	30.26	36.61	34.85	56.32	48.07
	Global	30.55	29.30	36.49	33.00	54.30	45.50
2nd-order	Layer	27.03	27.30	31.28	30.84	44.49	41.23
	Global	26.80	25.95	30.77	29.16	43.84	39.15

Results Figure 1a compares training-loss trajectories at 90% sparsity and reveals a clear ordering across saliency choices. Second-order saliency (orange) consistently attains the lowest loss throughout training and ends with a substantial margin, whereas Magnitude-based (blue) and First-order (green) saliency track closely and remain noticeably higher. Notably, all methods exhibit an early loss increase before stabilizing, but only the second-order criterion recovers to a markedly better regime. Overall, the plot suggests that incorporating curvature information leads to more reliable identification of critical parameters in the pretrained landscape, while purely magnitude- or gradient-based criteria are less effective at this extreme sparsity level.

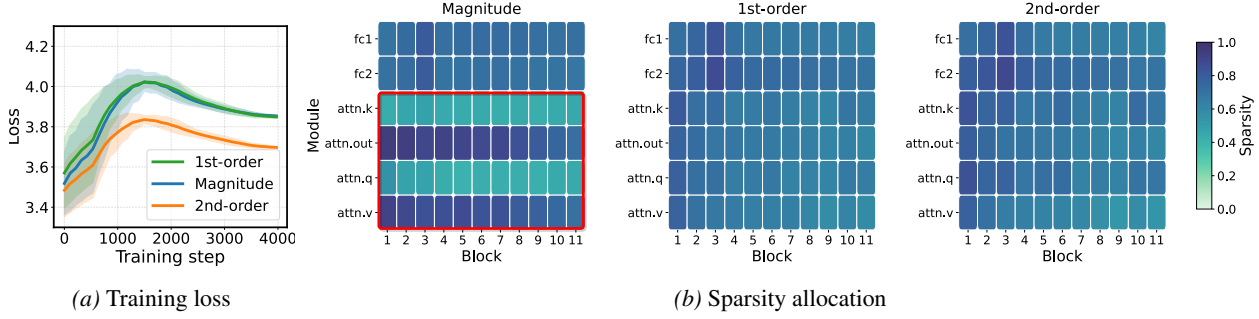


Figure 1. Effects of saliency criteria on (a) validation loss at 90% sparsity and (b) sparsity allocation under global pruning at 70% sparsity for OPT-125M. Validation loss curves are smoothed using a sliding-window average, and the shaded regions indicate the corresponding standard deviation.

This hierarchy is corroborated by the perplexity benchmarks in Table 1. Across all sparsity levels (50%–90%), second-order criteria consistently yield the lowest perplexity on WikiText-2 and C4, regardless of comparison group (i.e. Layer, Global), which will be discussed in the following subsection. This robustness, particularly in high-sparsity regimes (e.g., 90%), underscores that curvature-aware metrics provide the most reliable estimate of parameter importance, minimizing degradation regardless of the sparsity levels.

3.1.3. SPARSITY ALLOCATION ANALYSIS

Solving Equation (2) via global top- k selection enables sparsity to adapt to module sensitivity, overcoming the rigid uniformity enforced by standard layer-wise solvers (Frantar & Alistarh, 2023; Sun et al., 2024). We assess the benefits of this relaxation on OPT-125M in Table 1.

The results reveal a sharp divergence based on the saliency metric. Global pruning proves detrimental for Magnitude saliency, significantly degrading perplexity even at 50% sparsity (27.68 $\rightarrow$ 34.54 at C4). Conversely, derivative-based criteria consistently benefit from global relaxation; at 90% sparsity, global pruning reduces C4 perplexity by 2.57 and 2.08 for first- and second-order saliency, respectively. This indicates that lifting the uniformity constraint is highly beneficial, but strictly requires a saliency metric that accurately reflects global parameter importance.

To better understand this behavior, we visualize the resulting sparsity distributions at a 70% target sparsity in Figure 1b. The heatmap reveals that Magnitude-based pruning causes layer collapse, disproportionately stripping layers with naturally small weights (see red color outlined box of Figure 1b-Magnitude, where ‘attn.out’, ‘attn.v’ modules almost collapse (> 0.9) while attn.k, attn.q remains comparatively less pruned (< 0.5)). In contrast, derivative-based saliencies produce stable, non-uniform allocations—automatically assigning higher sparsity to redundant MLP layers while protecting sensitive attention mechanisms. This confirms

that effective non-uniformity does not require complex allocation heuristics (Yin et al., 2023; Sieberling et al., 2024; Huang et al., 2025); it can be achieved via simple global pruning, provided the underlying saliency is robust.

3.2. Sparsity Schedule

As we have discussed the design strategies to solve Equation (2), we now delve into sparsification schedule in Equation (3), where we study which sparsity schedule is beneficial for obtaining highly sparse LLMs.

3.2.1. ONE-SHOT SPARSIFICATION

A direct strategy for scheduling sparsity is to determine the pruning mask instantaneously, known as one-shot sparsification (i.e., $\|\mathbf{m}^0\|_0 = k$). This approach executes Equation (2) in a single step, effectively reducing the pruning problem to standard risk minimization with a fixed sparse topology ($\mathbf{m}^0 \odot \mathbf{w}$, Equation 4).

3.2.2. GRADUAL SPARSIFICATION

Another strategy to address Equation (3) is gradual sparsification, where the sparsity constraint is imposed incrementally during the training process. While this approach has proven effective for achieving extreme sparsity in traditional deep learning (Zhu & Gupta, 2017; Kurtic & Alistarh, 2022), it has not yet been studied at LLM scale. To investigate this, we adopt a cubic schedule to control the sparsity level S_t at timestep t :

$$S_t = S_{\text{final}} + (S_{\text{init}} - S_{\text{final}}) \left(1 - \frac{t - t_0}{T_{\text{prune}}}\right)^3 \quad (10)$$

where S_{init} and S_{final} represent the initial (typically 0) and final target sparsity, respectively.

Setup. To determine the optimal sparsity progression, we compare gradual sparsification strategy against standard one-shot baselines: Magnitude, Wanda (Sun et al., 2024),

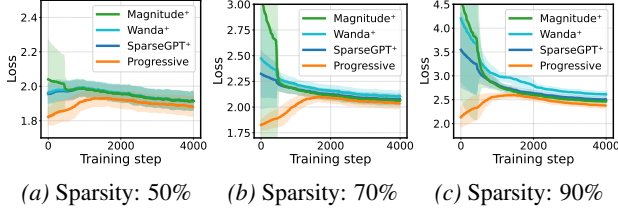


Figure 2. Training loss curves for LLaMA-2-7B comparing one-shot pruning followed by retraining and progressive sparsification across different sparsity levels.

Table 2. Effect of warm-up across sparsity levels on OPT-125M. For reference, the dense model achieves perplexities of 27.65 on WikiText and 26.56 on C4.

Warm-up	Sparsity					
	50%		70%		90%	
	Wikitext	C4	Wikitext	C4	Wikitext	C4
✗	28.95	27.93	33.14	31.04	45.06	39.91
✓	26.80	25.95	30.77	29.95	43.84	39.15

and SparseGPT (Frantar & Alistarh, 2023) on LLaMA-2-7B (Touvron et al., 2023), at {50%, 70%, 90%} sparsity. To ensure a fair comparison, all methods are allocated an identical retraining budget (denoted by the superscript $^+$).

For the gradual sparsification schedule, we update the pruning mask periodically rather than at every iteration to stabilize retraining. We perform $N = 50$ mask updates over a pruning phase of duration T_{prune} , yielding an update interval of $\Delta t = T_{\text{prune}}/N$. We set T_{prune} to 50% of the total training budget and use the remaining steps to retrain the masked model.

Result Figure 2 confirms that the necessity of gradual schedules scales with sparsity. As sparsity escalates from 50% (Figure 2a) to 90% (Figure 2c), one-shot baselines (Magnitude $^+$, Wanda $^+$, SparseGPT $^+$) exhibit increasingly catastrophic initial loss spikes. This demonstrates that abrupt one-shot pruning displaces the model into a high-loss region that is effectively unrecoverable; for instance, the initial loss for Magnitude $^+$ surges from 2.97 at 50% to 11.14 at 90%. Consequently, this early instability appears to hinder long-term convergence, as these methods struggle to fully recover from the initial degradation and result in consistently higher final loss values compared to our gradual strategy.

In contrast, the gradual approach (shown in red) mitigates this by inducing sparsity incrementally. By enforcing a slower progression of sparsity, our method effectively mitigates initial loss spikes and maintains the model in a stable optimization regime. This stability enables more effective recovery, ultimately yielding significantly lower final loss values (e.g., Wanda $^+$ = 2.62 vs. Ours = 2.37 at 90% sparsity). This empirically validates the superiority of gradual

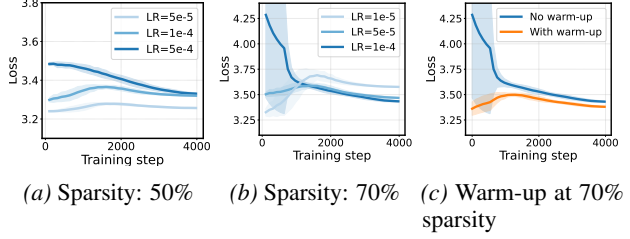


Figure 3. Validation loss curve under different learning rate configurations. (a) and (b): increasing sparsity leads to larger optimal learning rates but induces severe early-stage loss spikes. (c): learning rate warm-up effectively stabilizes training and mitigates optimization shock.

sparsification, confirming that spreading the pruning budget over time is beneficial for preventing performance degradation in high-sparsity regimes.

3.3. Training Strategy

Now we study the characteristics of Equation (4), where our goal is to effectively minimize the risk $\mathcal{L}$ with the remaining parameter ($m^{n+1} \odot w$), also known as sparse training. Sparse models often exhibit unstable gradient dynamics, typically necessitating higher learning rates to escape local minima (Frantar et al., 2023; Dey et al., 2024). This trend is corroborated by Figures 3a and 3b, where we observe a distinct upward shift in the optimal learning rate as sparsity increases (e.g., $5e-5$ at 50% $\rightarrow 1e-3$ at 70%). However, this presents a dilemma because applying aggressive learning rates immediately to a pretrained model causes a spike in initial loss, as seen in Figure 3a ($\text{LR} = 5e-4$), which potentially hurts the training procedure.

To reconcile these conflicting requirements of aggressive updates for recovery versus initial stability, we repurpose linear learning rate warmup (Goyal et al., 2017), a technique known to mitigate optimization instability during the initial training phase. We define this schedule as follows:

$$\eta_t = \begin{cases} \eta_{\text{target}} \cdot \frac{t}{T_{\text{warmup}}} & \text{if } t \leq T_{\text{warmup}} \\ \eta_{\text{target}} \cdot \frac{T-t}{T-T_{\text{warmup}}} & \text{if } t > T_{\text{warmup}} \end{cases}, \quad (11)$$

where η_t denotes learning rate at step t , and T denotes the total number of training steps. This strategy operates in synergy with our gradual sparsity schedule (Equation (10)). By slowly ramping up the learning rate as sparsity is introduced, we dampen destabilizing gradient updates during critical structural shifts.

Figure 3c validates this strategy, demonstrating that warm-up effectively suppresses the initial instability caused by applying high learning rates to pretrained weights. By preventing the early loss spike, warm-up allows the model to safely reach the aggressive learning rate magnitudes required for recovery, subsequently leveraging the decay

Table 3. Perplexity ($\downarrow$) and Average Zero-Shot accuracy (AZS, %, $\uparrow$) of LLaMA-2-7B across moderate sparsity levels for different pruning methods. Superscript $^+$ denotes results after one-shot pruning followed by retraining. The top panel contains methods without retraining, whereas the bottom panel contains retraining-based methods. The cells with high degradation in performance are shaded by its magnitude (the worse, the darker). For reference, the dense model achieves perplexities of 5.47 on WikiText and 7.26 on C4, and AZS of 59.67%.

Method	Sparsity								
	50%			70%			90%		
	Wiki $\downarrow$	C4 $\downarrow$	AZS $\uparrow$	Wiki $\downarrow$	C4 $\downarrow$	AZS $\uparrow$	Wiki $\downarrow$	C4 $\downarrow$	AZS $\uparrow$
Magnitude	16.03	21.34	51.14	5.0e4	2.8e4	33.43	NaN	NaN	32.40
Wanda	6.92	9.24	55.78	76.32	81.08	35.36	2.0e4	1.0e4	31.80
SparseGPT	7.01	9.23	56.31	27.12	30.94	41.39	1430	864.5	32.46
L-ADMM	6.80	8.97	57.73	20.56	22.20	44.22	400.5	287.1	31.88
ALPS	6.86	9.02	57.14	19.39	20.37	45.03	248.8	180.9	32.03
SAFE	6.72	8.87	55.30	86.80	48.54	38.94	1.6e4	1.6e4	32.63
SparseLLM	7.23	9.51	55.63	37.65	35.00	40.54	1267	648.0	31.93
ELSA	7.5	9.81	56.39	13.20	14.08	46.21	26.97	23.14	38.52
Magnitude $^+$	6.41	8.83	55.80	7.72	10.04	52.18	13.18	15.40	40.58
Wanda $^+$	6.41	8.51	56.61	8.06	10.44	49.36	15.61	18.82	38.34
SparseGPT $^+$	6.38	8.46	57.08	7.74	10.07	51.00	13.41	16.04	39.57
BEST	6.20	8.19	57.78	7.52	9.56	52.06	11.60	13.71	42.34

phase for precise convergence. This optimization stability translates directly to language modeling performance; as shown in Table 2, incorporating warmup consistently improves perplexity across all sparsity levels, confirming that this simple strategy is effective for high-sparsity recovery.

3.4. Summary Recipe

In this section, we have systematically dissected the components required for extreme LLM compression. While prior LLM pruning literature has largely overlooked these factors in favor of one-shot design, we demonstrate that saliency, scheduling, and training strategies are not merely implementation details but governing factors when pruning to extreme sparsity. Synthesizing our empirical findings, we propose the following optimal configuration for highly sparse LLMs:

- **Approximated second-order global masking.** Extreme sparsity necessitates second-order saliency, whereas magnitude is sufficient for progressive sparsity in vision models. Saliency is crucial for globally pruning, second-order can bypassing manual sparsity allocation across different layers.
- **Gradual sparsification.** One-shot pruning induces irreversible damage in extreme sparsity; a gradual decay is beneficial to stabilize the loss landscape and allow better optimization.
- **Linear learning rate warmup (+ decay).** Learning rate warmup mitigates the initial spike of loss at the beginning of the training pretrained models.

We collectively refer to this optimized pipeline as BEST for brevity. In the following section, we demonstrate that this recipe enables LLaMA-2-7B to retain significant performance even at 99% sparsity, a regime previously considered

intractable.

4. Validation

We evaluate our complete framework against prior approaches on standard language modeling and inference efficiency benchmarks. Precisely, we sparsify LLaMA-2-7B (Touvron et al., 2023), an open-source LLM widely used as standard benchmark in the literature, from moderate 50% to extreme 99% sparsity, and measure the final language modeling performance (perplexity), zero-shot generalization performance (accuracy, %), and efficiency metrics such as inference acceleration and memory savings.

Comparative Methods We employ wide range of representative baseline methods: magnitude pruning (Han et al., 2015), Wanda (Sun et al., 2024), SparseGPT (Frantar & Alistarh, 2023), L-ADMM (Boža, 2024), ALPS (Meng et al., 2024), SAFE (Lee et al., 2025), SparseLLM (Bai et al., 2024), ELSA (Lee et al., 2026). We also compare with three one-shot+retrain baselines (denoted with the superscript $^+$) for Magnitude, Wanda, and SparseGPT under the same retraining budget as BEST. Full training details and hyperparameter configurations are provided in Appendix B.2.

4.1. Language Modeling Performance

Tables 3 and 4 reports the perplexity evaluations on WikiText-2 (Wiki) and C4, where our method consistently outperforms all baselines. In moderate sparsity regimes ($\leq 90\%$), we observe a clear hierarchy among one-shot baselines, where SparseGPT $^+$ consistently achieves the lowest perplexity, followed by Wanda $^+$ and Magnitude $^+$.

However, a performance gap becomes larger in the extreme

Table 4. Perplexity ($\downarrow$) and Average Zero-Shot accuracy (AZS, %, $\uparrow$) of LLaMA-2-7B at extreme sparsity levels for retraining-based methods.

Method	Sparsity					
	Wiki $\downarrow$	95% C4 $\downarrow$	AZS $\uparrow$	Wiki $\downarrow$	99% C4 $\downarrow$	AZS $\uparrow$
Magnitude ⁺	20.52	21.50	37.87	50.40	45.02	36.48
Wanda ⁺	21.87	22.47	37.72	44.52	38.61	37.21
SparseGPT ⁺	<u>18.75</u>	<u>20.72</u>	<u>37.94</u>	<u>35.53</u>	<u>32.39</u>	<u>36.59</u>
BEST	13.48	16.52	39.71	19.67	22.03	37.84

sparsity regime (95%–99%). As sparsity increases, these one-shot methods suffer higher degradation, struggling to recover from the initial pruning step even with retraining. This disparity is particularly evident at 99% sparsity, where our method achieves a WikiText-2 perplexity of 19.67, effectively halving the perplexity of the strongest baseline, SparseGPT⁺ (35.53). These results demonstrate that standard retraining pipelines are insufficient for extreme compression, validating the necessity of the comprehensive stabilization recipe proposed in this work.

4.2. Zero-shot Generalization Performance

We evaluate zero-shot performance on downstream tasks to verify generalization capabilities. Our method consistently achieves the highest average accuracy across all sparsity levels (AZS (%) on Tables 3 and 4). Notably, at 90% sparsity, we outperform the strongest baseline (SparseGPT⁺) by nearly 3% on average (42.34% vs. 39.57%). This advantage persists into the extreme regime in Table 10; at 99% sparsity, our method retains 40.40% accuracy on ARC-E, significantly surpassing SparseGPT⁺ (33.67%) and Wanda⁺ (34.47%). While all methods struggle at this limit, our framework demonstrates better retention of task performance compared to standard retraining, which aligns with the results in language modeling performance.

Table 5. Memory savings and end-to-end inference accelerations (Latency (s), Tokens/s) of sparse LLaMA-2-7B.

	Dense	70%	90%	95%
Latency (s)	1.84	1.00 ($\times 1.84$)	0.65 ($\times 2.83$)	0.57 ($\times 3.22$)
Tokens/s	54.47	100.17 ($\times 1.84$)	154.31 ($\times 2.83$)	176.18 ($\times 3.23$)
Memory (GB)	13.6	5.74 ($\times 2.37$)	2.84 ($\times 4.79$)	2.19 ($\times 6.21$)

4.3. Practical Efficiency

To translate theoretical parameter reduction into tangible system acceleration, we utilize the MACKO kernel (Macko & Boža, 2025). We integrate the MACKO-SpMV kernel into the inference pipeline of our pruned LLaMA-2-7B models to measure three key metrics: end-to-end latency (per-token decoding time), generation throughput (tokens/sec), and peak memory consumption (GB) during inference, measured on a single RTX3090 GPU.

Table 5 summarizes these metrics, comparing the dense baseline against our models compressed to high sparsity. We can observe that sparsity directly translates into substantial performance gains. As detailed in the table, moving into the high-sparsity regime yields non-linear improvements in efficiency. Notably, at 95% sparsity, the model achieves a **3.23 $\times$** increase in generation throughput (176.18 tokens/s vs. 54.47 tokens/s) and a **6.21 $\times$** reduction in peak memory usage (dropping from 13.6GB to 2.18GB). These results validate that our pruning framework not only preserves model quality but also unlocks deployment capabilities on consumer-grade hardware that were previously inaccessible for dense LLMs.

5. Conclusion

In this work, we demonstrate that current state of LLM pruning practices has led to producing much under-optimized sparse language models. Our systematic investigation reveals that a simple progressive sparse training with careful choice of second-order saliency, gradual sparsity scheduling, and training stabilization easily outperforms all existing baselines to set a new state of the art and produces extremely sparse models that prior studies fail to obtain. Furthermore, we showed that these theoretical gains translate into tangible system benefits, achieving over 3.22 $\times$ inference acceleration and 6.21 $\times$ memory reduction via modern sparse kernels. We interpret these results as an update in the state of sparsity for LLMs, indicating that substantial opportunities for pruning research remain unexplored.

Limitations and Future Work While our framework establishes a robust recipe for extreme compression, several avenues remain for exploration. First, we have primarily focused on the progressive sparsification paradigm (Zhu & Gupta, 2017). Future work could extend this by incorporating dynamic sparsity mechanisms (Evci et al., 2020; Liu et al., 2020), or exploiting knowledge distillation (Muralidharan et al., 2024). Such approaches may further enhance plasticity and improve performance in extreme sparsity regime. Second, we primarily analyze inference efficiency, whereas progressive sparse training can be utilized for efficient training as well (Agarwalla et al., 2024). Future work will focus on leveraging this extreme sparsity to accelerate training and parallel operations, aiming to reduce the computational cost of the sparsification process itself and further bridge the gap between sparse and dense modeling paradigms.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. Our research focuses on enabling extreme sparsity in LLMs, which directly contributes to re-

ducing the computational resources and energy consumption required for model training and inference. By mitigating the resource requirements of LLMs, this work also has the potential to democratize access to powerful AI models on resource-constrained devices. There are no specific negative societal consequences that we feel must be highlighted here.

References

- Agarwalla, A., Gupta, A., Marques, A., Pandit, S., Goin, M., Kurtic, E., Leong, K., Nguyen, T., Salem, M., Alistarh, D., et al. Enabling high-sparsity foundational llama models with efficient pretraining and deployment. *arXiv*, 2024.
- Bai, G., Li, Y., Ling, C., Kim, K., and Zhao, L. Sparsellm: Towards global pruning of pre-trained language models. *NeurIPS*, 2024.
- Beck, A. and Teboulle, M. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM*, 2009.
- Botev, A., Ritter, H., and Barber, D. Practical gauss-newton optimisation for deep learning. In *International Conference on Machine Learning*, pp. 557–565. PMLR, 2017.
- Boža, V. Fast and effective weight update for pruned large language models. *TMLR*, 2024.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *NeurIPS*, 2020.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. *NAACL*, 2019.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Dey, N., Bergsma, S., and Hestness, J. Sparse maximal update parameterization: A holistic approach to sparse training dynamics. *NeurIPS*, 2024.
- Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. Rigging the lottery: Making all tickets winners. *ICML*, 2020.
- Fan, R., Yu, X., Dong, P., Li, Z., Gong, G., Wang, Q., Wang, W., and Chu, X. Spinfer: Leveraging low-level sparsity for efficient large language model inference on gpus. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 243–260, 2025.
- Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.
- Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot. 2023.
- Frantar, E., Riquelme, C., Houlsby, N., Alistarh, D., and Evci, U. Scaling laws for sparsely-connected foundation models. *ICLR*, 2023.
- Gale, T., Zaharia, M., Young, C., and Elsen, E. Sparse gpu kernels for deep learning. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14. IEEE, 2020.
- Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., and He, K. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.
- Gray, S., Radford, A., and Kingma, D. P. Gpu kernels for block-sparse weights. *arXiv preprint arXiv:1711.09224*, 3(2):2, 2017.
- Hagiwara, M. A simple and effective method for removal of hidden units and weights. *Neurocomputing*, 1994.
- Han, S., Pool, J., Tran, J., and Dally, W. Learning both weights and connections for efficient neural network. *NeurIPS*, 2015.
- Hassibi, B. and Stork, D. Second order derivatives for network pruning: Optimal brain surgeon. *NeurIPS*, 1992.
- Hoeftler, T., Alistarh, D., Ben-Nun, T., Dryden, N., and Peste, A. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *JMLR*, 2021.
- Huang, W., Zhang, Y., Zheng, X., Chao, F., and Ji, R. Determining layer-wise sparsity for large language models through a theoretical perspective. *arXiv preprint arXiv:2502.14770*, 2025.
- Janowsky, S. A. Pruning versus clipping in neural networks. *Physical Review A*, 1989.
- Kim, B.-K., Kim, G., Kim, T.-H., Castells, T., Choi, S., Shin, J., and Song, H.-K. Shortened llama: Depth pruning for large language models with comparison of retraining methods. *arXiv preprint arXiv:2402.02834*, 2024.
- Kingma, D. P. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Kunstner, F., Hennig, P., and Balles, L. Limitations of the empirical fisher approximation for natural gradient descent. *Advances in neural information processing systems*, 32, 2019.

- Kurtic, E. and Alistarh, D. Gmp*: Well-tuned gradual magnitude pruning can outperform most bert-pruning methods. *arXiv preprint arXiv:2210.06384*, 2022.
- LeCun, Y., Denker, J., and Solla, S. Optimal brain damage. *NeurIPS*, 1989.
- Lee, D., Lee, K., Chung, J., and Lee, N. Safe: Finding sparse and flat minima to improve pruning. *arXiv preprint arXiv:2506.06866*, 2025.
- Lee, K., Jang, H., Lee, D., Alistarh, D., and Lee, N. The unseen frontier: Pushing the limits of llm sparsity with surrogate-free admm. *ICLR*, 2026.
- Lee, N., Ajanthan, T., and Torr, P. H. Snip: Single-shot network pruning based on connection sensitivity. *ICLR*, 2019.
- Lie, S. Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning. *IEEE Micro*, 43(3):18–30, 2023.
- Liu, J., Xu, Z., Shi, R., Cheung, R. C., and So, H. K. Dynamic sparse training: Find efficient sparse network from scratch with trainable masked layers. *arXiv preprint arXiv:2005.06870*, 2020.
- Lu, Y. and Liu, W. Dasp: Specific dense matrix multiply-accumulate units accelerated general sparse matrix-vector multiplication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2023.
- Macko, V. and Boža, V. Macko: Sparse matrix-vector multiplication for low sparsity. *ArXiv*, 2025.
- Martens, J. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.
- Meng, X., Behdin, K., Wang, H., and Mazumder, R. Alps: Improved optimization for highly sparse one-shot pruning for large language models. *NeurIPS*, 2024.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *ICLR*, 2017.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. *EMNLP*, 2018.
- Mishra, A., Latorre, J. A., Pool, J., Stosic, D., Stosic, D., Venkatesh, G., Yu, C., and Micikevicius, P. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021.
- Mocanu, D. C., Mocanu, E., Stone, P., Nguyen, P. H., Gibescu, M., and Liotta, A. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature communications*, 9(1):2383, 2018.
- Mozer, M. C. and Smolensky, P. Skeletonization: A technique for trimming the fat from a network via relevance assessment. *NeurIPS*, 1988.
- Muralidharan, S., Turuvekere Sreenivas, S., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Compact language models via pruning and knowledge distillation. *Advances in Neural Information Processing Systems*, 37:41076–41102, 2024.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Peste, A., Iofinova, E., Vladu, A., and Alistarh, D. Ac/dc: Alternating compressed/decompressed training of deep neural networks. *NeurIPS*, 2021.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 2020.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Sanh, V., Wolf, T., and Rush, A. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in neural information processing systems*, 33:20378–20389, 2020.
- Shin, S., Park, W., Lee, J., and Lee, N. Rethinking pruning large language models: Benefits and pitfalls of reconstruction error minimization. 2024.
- Sieberling, O., Kuznedelev, D., Kurtic, E., and Alistarh, D. Evopress: Towards optimal dynamic model compression via evolutionary search. *arXiv preprint arXiv:2410.14649*, 2024.
- Singh, S. P. and Alistarh, D. Woodfisher: Efficient second-order approximation for neural network compression. *NeurIPS*, 2020.
- Soboleva, D., Al-Khateeb, F., Myers, R., Steeves, J. R., Hestness, J., and Dey, N. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://www.cerebras.net/blog/>

- slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama, 2023. URL <https://huggingface.co/datasets/cerebras/SlimPajama-627B>.
- Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. 2024.
- Sung, Y.-L., Yoon, J., and Bansal, M. Ecoflap: Efficient coarse-to-fine layer-wise pruning for vision-language models. *arXiv preprint arXiv:2310.02998*, 2023.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Xia, H., Zheng, Z., Li, Y., Zhuang, D., Zhou, Z., Qiu, X., Li, Y., Lin, W., and Song, S. L. Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *arXiv preprint arXiv:2309.10285*, 2023.
- Yin, L., Wu, Y., Zhang, Z., Hsieh, C.-Y., Wang, Y., Jia, Y., Li, G., Jaiswal, A., Pechenizkiy, M., Liang, Y., et al. Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity. *arXiv preprint arXiv:2310.05175*, 2023.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *ACL*, 2019.
- Zeng, W. and Urtasun, R. Mlprune: Multi-layer pruning for automated neural network compression. *arXiv*, 2018.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- Zhou, A., Ma, Y., Zhu, J., Liu, J., Zhang, Z., Yuan, K., Sun, W., and Li, H. Learning n:m fine-grained structured sparse neural networks from scratch, 2021.
- Zhu, M. and Gupta, S. To prune, or not to prune: exploring the efficacy of pruning for model compression. *arXiv*, 2017.

A. Additional Results

B. Experimental Details

B.1. Implementation and Reproducibility Details

Our experiments are implemented in PyTorch (Paszke et al., 2019), with model architectures and datasets managed through the HuggingFace `transformers` and `datasets` libraries. Sparse training is implemented on top of the HuggingFace `Trainer` framework and supports distributed execution via PyTorch FSDP-2 (Zhao et al., 2023), integrated with HuggingFace Accelerate.

All experiments are conducted using a single unified codebase to ensure fair and consistent comparisons. Whenever available, baseline methods are reproduced using their official implementations. To facilitate reproducibility, we will publicly release the full experimental setup, including dependency versions, configuration files, and training scripts.

Experiments are executed on NVIDIA A6000, A100, and PRO6000 GPUs. Computational resources are scaled with model size: LLaMA-2-7B is trained using 2–4 NVIDIA A100 GPUs depending on the experimental configuration, while smaller models such as OPT-125M are trained on a single GPU.

B.2. Experimental setup for Section 3, Section 4

Calibration Data To perform one-shot pruning for Wanda⁺ and SparseGPT⁺, we follow the standard one-shot pruning protocol of Frantar & Alistarh (2023). Specifically, we sample 128 calibration sequences from the C4 dataset with a sequence length of 2048, which are used solely for one-shot pruning.

Training Data and Details. We conduct all experiments using the SlimPajama² dataset, which is a cleaned and deduplicated version of RedPajama comprising high-quality pre-training corpora from diverse sources including C4, ArXiv, and GitHub. For both our proposed strategy and the one-shot+retrain strategy, we construct training data by randomly sampling non-overlapping token sequences of length 2,048. To ensure a fair and controlled comparison across different settings, we fix the total number of training tokens to $64 \times 4000 \times 2048$, where 64 denotes the batch size, 4000 the total number of training steps, and 2048 the token sequence length. All model parameters and optimizer states use full precision for training, and automatic mixed precision with bf16 precision is used for efficient training. A full list of hyperparameter configurations is provided in Tables 6 to 8.

Table 6. Global hyperparameters of BEST and one-shot+retrain strategy shared across all models.

Hyperparameter	BEST	one-shot+retrain
LR schedule	Linear decay	Linear decay
Pruning interval k	40	–
Adam (β_1, β_2)	(0.9, 0.999)	(0.9, 0.999)
Batch size	64	64
Training steps	4000	4000
Warm-up steps	400	400

Table 7. Learning rate (η) configurations of BEST across sparsity levels.

Sparsity	OPT-125M	LLaMA-2-7B
50%	1e−4	5e−5
70%	5e−4	5e−5
90%	1e−3	1e−4
95%	–	5e−4
99%	–	5e−4

²<https://huggingface.co/datasets/DKYoon/SlimPajama-6B>

Table 8. Learning rate (η) configurations of one-shot+retrain strategy across sparsity levels.

Sparsity	LLaMA-2-7B
50%	1e-4
70%	1e-4
90%	1e-4
95%	5e-4
99%	5e-4

Evaluation We evaluate perplexity on held-out validation sets from C4 (Raffel et al., 2020) and Wikitext2 (Merity et al., 2017). For zero-shot evaluation, we use `lm-eval-harness` to assess performance on seven widely used benchmarks: ARC-Easy and ARC-Challenge (ARC-E/C) (Clark et al., 2018), BoolQ (Clark et al., 2019), HellaSwag (Zellers et al., 2019), OpenBookQA (OBQA) (Mihaylov et al., 2018), RTE (Zeng & Urtasun, 2018), and Winogrande (Sakaguchi et al., 2021). We report the mean accuracy across tasks, following the evaluation protocol in Section 4.

C. Ablation Study

C.1. Diagonal Fisher Approximation via Adam Second Moment

We conduct ablation studies on OPT-125M to validate the effectiveness of reusing the Adam optimizer state for pruning. To compute the saliency score for each parameter w_i , we contrast two strategies: directly reusing the accumulated second-moment element $v_{t,i}$ versus recomputing the empirical Fisher $\mathbb{E}[g_i^2]$ using gradients collected from 128, 512, and 2048 sequences.

Table 9 presents the perplexity on WikiText-2 and C4 datasets across sparsity levels of 50%, 70%, and 90%. The results demonstrate that recomputing the second-moment estimate yields no consistent performance improvement over simple state reuse. Even with 2048 calibration sequences, the perplexity differences remain marginal across all sparsity regimes. Considering that recomputing $\mathbb{E}[g_i^2]$ introduces non-trivial computational overhead due to additional forward-backward passes, we conclude that reusing the existing Adam state provides a sufficiently accurate and efficient proxy for curvature in gradual pruning.

Table 9. Effect of reusing Adam optimizer state for diagonal Fisher approximation. “Recompute” indicates whether the second-moment state is recomputed before pruning, and $|D|$ denotes the size of the calibration dataset.

Recompute	$ D $	Sparsity					
		50%		70%		90%	
		Wiki	C4	Wiki	C4	Wiki	C4
×	–	26.80	25.95	30.77	29.95	<u>43.84</u>	39.15
✓	128	27.02	26.23	31.32	<u>29.61</u>	44.51	39.76
	512	26.90	26.16	<u>30.96</u>	29.54	43.79	39.39
	2048	<u>26.89</u>	<u>26.12</u>	31.17	29.62	43.96	<u>39.24</u>

D. Additional Results

D.1. Sparsity Allocation

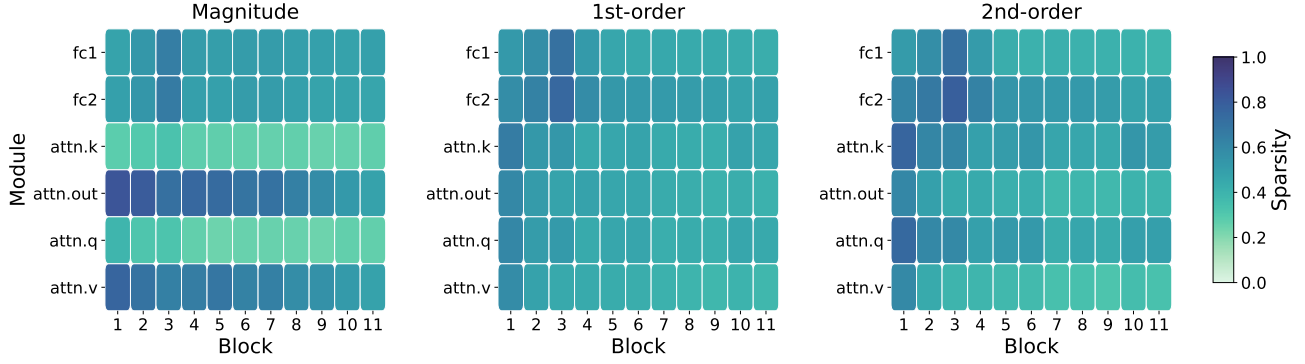


Figure 4. Sparsity allocation heatmaps for OPT-125M pruned to 50% sparsity.

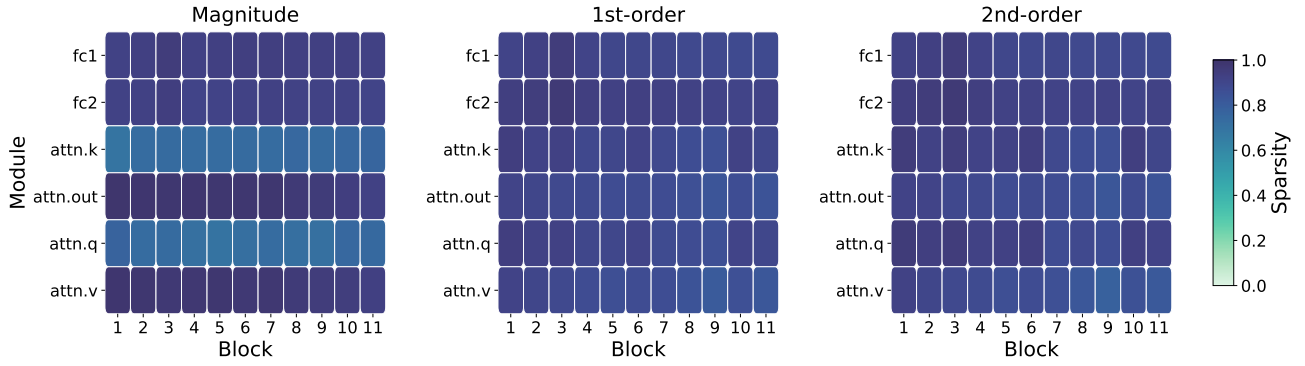


Figure 5. Sparsity allocation heatmaps for OPT-125M pruned to 90% sparsity.

Table 10. Zero-shot accuracy (%) of LLaMA-2-7B across multiple tasks under different sparsity regimes. Higher values indicate better performance.

Sparsity	Method	ARC-C	ARC-E	BoolQ	HellaSwag	OBQA	RTE	Winogrande	Avg
0%	Dense	43.35	76.26	77.68	57.14	31.40	62.82	69.06	59.67
50%	Magnitude	34.90	64.02	62.91	49.13	26.80	57.04	63.22	51.14
	Wanda	39.25	72.22	75.17	52.64	30.60	53.43	67.17	55.78
	SparseGPT	38.23	71.34	<u>75.99</u>	52.70	29.80	56.32	69.77	56.31
	L-ADMM	39.68	72.77	76.24	53.35	31.40	61.37	<u>69.30</u>	57.73
	ALPS	<u>40.61</u>	<u>72.90</u>	75.44	53.37	30.80	57.76	68.98	57.14
	SAFE	38.14	72.14	74.83	52.15	26.00	57.04	66.77	55.30
	SparseLLM	38.05	71.25	75.14	52.66	29.60	53.43	<u>69.30</u>	55.63
	ELSA	39.42	71.30	73.03	53.12	29.40	58.48	66.54	56.39
	Magnitude ⁺	38.91	71.97	72.29	53.63	29.00	58.12	66.69	55.80
	Wanda ⁺	37.88	71.21	72.17	<u>53.86</u>	28.80	66.43	65.90	56.61
	SparseGPT ⁺	39.51	72.14	73.55	53.82	29.80	<u>64.62</u>	66.14	57.08
	BEST	40.91	73.57	73.98	54.77	<u>31.00</u>	61.01	67.80	<u>57.58</u>
70%	Magnitude	22.87	27.82	37.95	25.90	17.20	53.07	49.25	33.43
	Wanda	18.6	30.01	57.28	28.04	12.0	52.71	48.86	35.36
	SparseGPT	22.01	42.34	65.14	33.04	16.8	52.71	57.7	41.39
	L-ADMM	23.81	50.63	63.21	36.57	20.40	54.15	60.77	44.22
	ALPS	25.51	52.78	63.46	37.54	20.8	53.43	61.72	45.03
	SAFE	24.23	45.62	43.76	34.74	18.40	52.71	53.12	38.94
	SparseLLM	20.90	40.32	61.87	32.74	16.0	54.51	57.46	40.54
	ELSA	27.13	55.81	63.61	43.16	22.40	52.71	58.64	46.21
	Magnitude ⁺	32.08	<u>66.41</u>	<u>68.01</u>	48.73	26.80	59.93	<u>63.30</u>	52.18
	Wanda ⁺	30.55	63.68	65.11	46.45	22.60	<u>57.04</u>	60.06	49.36
	SparseGPT ⁺	<u>32.94</u>	64.23	69.24	48.47	25.40	53.07	63.69	51.00
	BEST	34.30	68.77	67.98	49.67	<u>25.60</u>	55.57	62.51	<u>52.06</u>
90%	Magnitude	22.78	25.93	39.17	25.53	16.0	47.29	50.12	32.40
	Wanda	21.67	25.46	37.83	25.83	15.2	47.29	49.33	31.80
	SparseGPT	20.65	26.77	37.83	25.7	13.0	52.71	50.59	32.46
	L-ADMM	19.97	26.14	37.83	26.46	13.60	51.62	47.51	31.88
	ALPS	19.45	26.89	37.8	26.81	12.8	53.79	46.65	32.03
	SAFE	<u>21.84</u>	26.52	37.83	25.91	15.80	52.71	47.83	32.63
	SparseLLM	20.56	25.72	37.83	25.94	13.8	52.71	46.96	31.93
	ELSA	18.52	41.33	57.25	31.54	16.60	52.71	51.70	38.52
	Magnitude ⁺	20.48	48.27	58.07	33.82	<u>19.20</u>	52.71	51.54	40.58
	Wanda ⁺	18.94	42.63	58.78	30.49	15.40	<u>53.07</u>	49.09	38.34
	SparseGPT ⁺	19.03	45.12	<u>59.69</u>	<u>31.86</u>	16.40	52.71	2.17	39.57
	BEST	20.48	49.49	60.95	36.51	22.4	52.71	53.83	42.34
95%	Magnitude ⁺	<u>18.09</u>	39.31	60.64	28.15	15.20	52.35	<u>51.38</u>	37.87
	Wanda ⁺	17.58	36.9	<u>61.65</u>	27.94	15.00	52.71	52.25	37.72
	SparseGPT ⁺	17.92	<u>39.35</u>	61.83	<u>28.45</u>	<u>16.60</u>	52.35	49.09	<u>37.94</u>
	BEST	19.63	46.63	57.83	32.72	17.80	52.71	50.67	39.71
99%	Magnitude ⁺	16.98	34.39	61.96	26.95	11.60	51.26	52.25	36.48
	Wanda ⁺	18.34	34.47	<u>61.53</u>	26.91	14.6	<u>53.07</u>	51.54	<u>37.21</u>
	SparseGPT ⁺	<u>17.58</u>	33.67	60.09	<u>27.27</u>	13.20	53.79	50.51	36.59
	BEST	<u>17.58</u>	40.19	61.41	28.43	<u>13.60</u>	<u>53.07</u>	50.59	37.84